操作系统原理 lab1 实验报告

实验课程:操作系统原理

实验名称:**编译内核/利用已有内核构建OS**

专业名称:计算机科学与技术

学生姓名:梁力航

学生学号:23336128

实验地点:东实验楼 B201

实验成绩:

报告时间:

1. 实验要求

1. 搭建OS内核开发环境包括：代码编辑环境、编译环境、运行环境、调试环境等。
2. 下载并编译i386（32位）内核，并利用qemu启动内核。
3. 制作initramfs，熟悉方法。
4. 编写简单应用程序随内核启动运行。
5. 编译i386版本的Busybox，随内核启动，构建简单的OS。
6. 开启远程调试功能，进行调试跟踪代码运行。

2. 实验过程

环境配置

由于先前已配置过 Ubuntu 双系统，本段略过

配置C/C++环境

c++的头文件 include 时报错，采用下面网页的办法成功解决

https://blog.csdn.net/pjie131_/article/details/144710330?ops_request_misc=%257B%2522request%255Fid%2522%253A%2522baf0e54f0422b608eab8e7983fd2de51%2522%task-blog-2~all~sobaiduend~default-1-144710330-null-null.142

关键命令：

`sudo apt install gcc-multilib`

安装其他工具

安装gcc lib 时按照 readme 文件上的命令

```
sudo apt install nasm
sudo apt install qemu
sudo apt install cmake
sudo apt install libncurses5-dev
sudo apt install bison
sudo apt install flex
sudo apt install libssl-dev
sudo apt install libc6-dev-i386
```

报错依赖项约束，无法安装

查阅 CSDN 后，输入下面的命令解决

`sudo apt-get install lib32gcc-11-dev`

这行命令的目的安装匹配 gcc 版本（11）的 lib

编译Linux内核

下载内核

编译内核

启动内核并调试

启动qemu

`qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -s -S -append "console=ttyS0" -nographic`

gdb调试

在另外一个Terminal下启动gdb

```
cd ~/lab1
gdb
```

在gdb下，加载符号表

```
file linux-5.10.19/vmlinux
```

在gdb下，连接已经启动的qemu进行调试。

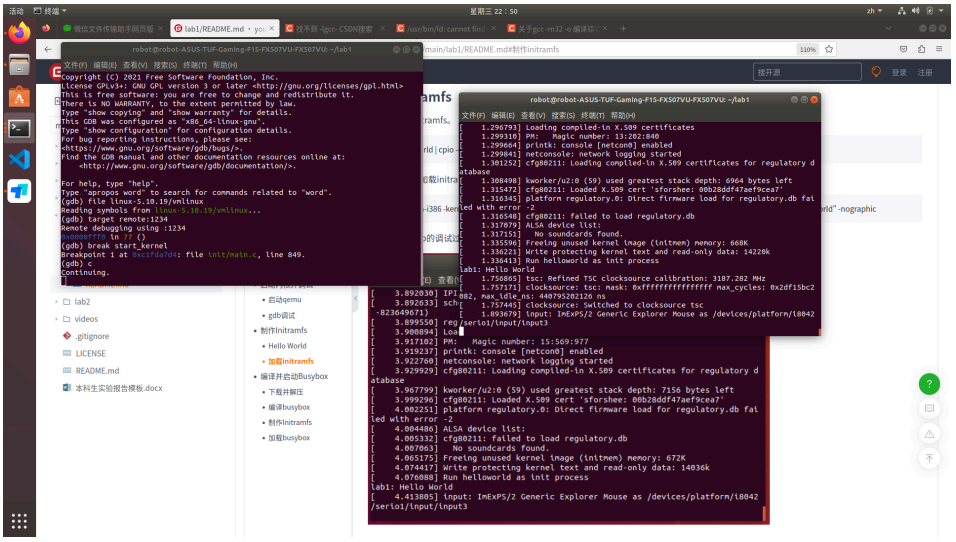
```
target remote:1234
```

在gdb下，为start_kernel函数设置断点。

```
break start_kernel
```

在gdb下，输入`c`运行。

成功：



制作Initramfs

Hello World

创建.c文件并编译

加载initramfs

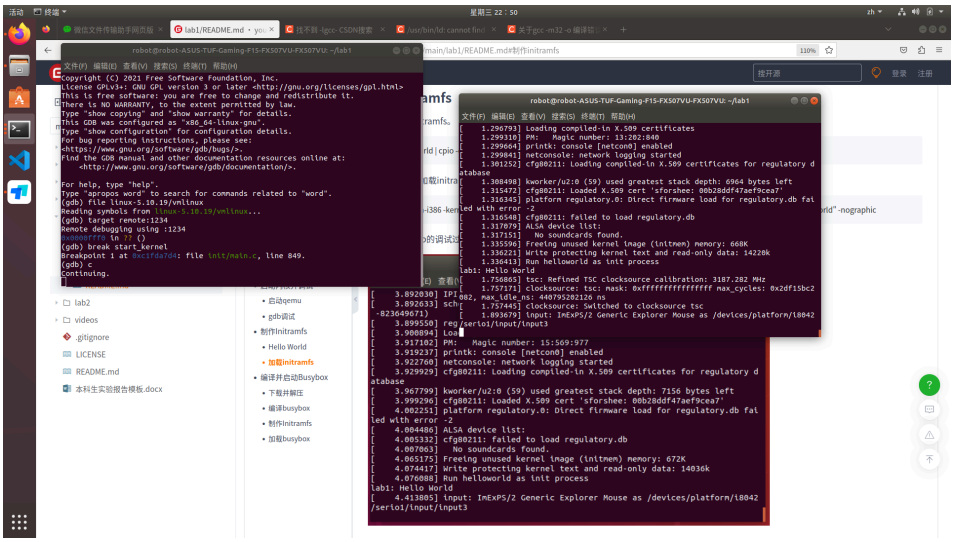
用cpio打包initramfs。

```
echo helloworld | cpio -o --format=newc > hwinitramfs
```

启动内核，并加载initramfs。

`qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -initrd hwinitramf`

可以看到qemu中输出了 `lab1: Hello World\n`



编译并启动Busybox

下载并编译 Busybox

制作Initramfs

将安装在_install目录下的文件和目录取出放在 `~/lab1/mybusybox` 处。

```
cd ~/lab1
mkdir mybusybox
mkdir -pv mybusybox/{bin,sbin,etc,proc,sys,usr/{bin,sbin}}
cp -av busybox-1_33_0/_install/* mybusybox/
cd mybusybox
```

initramfs需要一个init程序，可以写一个简单的shell脚本作为init。

用vim打开文件 `init`

```
vim init
```

复制入如下内容，保存退出。

```
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
echo -e "\nBoot took $(cut -d' ' -f1 /proc/uptime) seconds\n"
exec /bin/sh
```

加上执行权限。

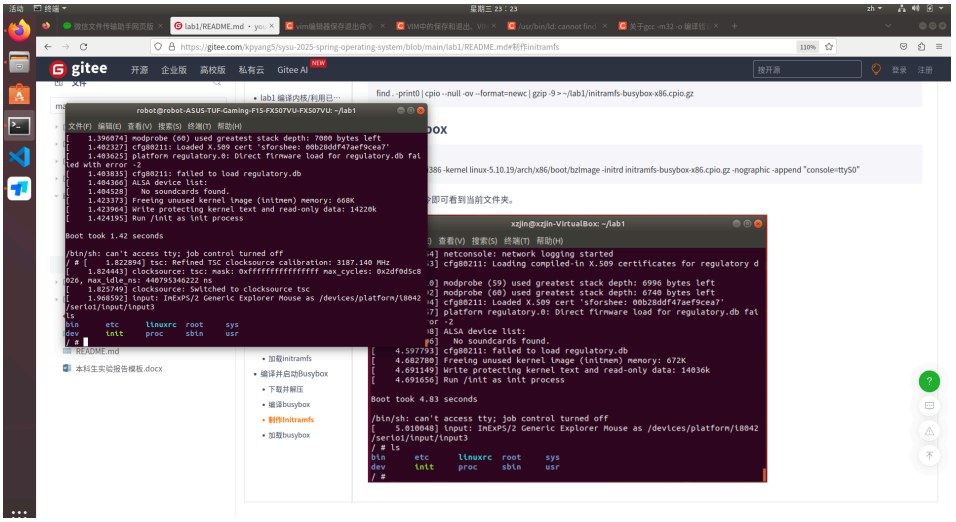
```
chmod u+x init
```

将mybusybox中的内容打包归档成cpio文件，以供Linux内核做initramfs启动执行。

加载busybox

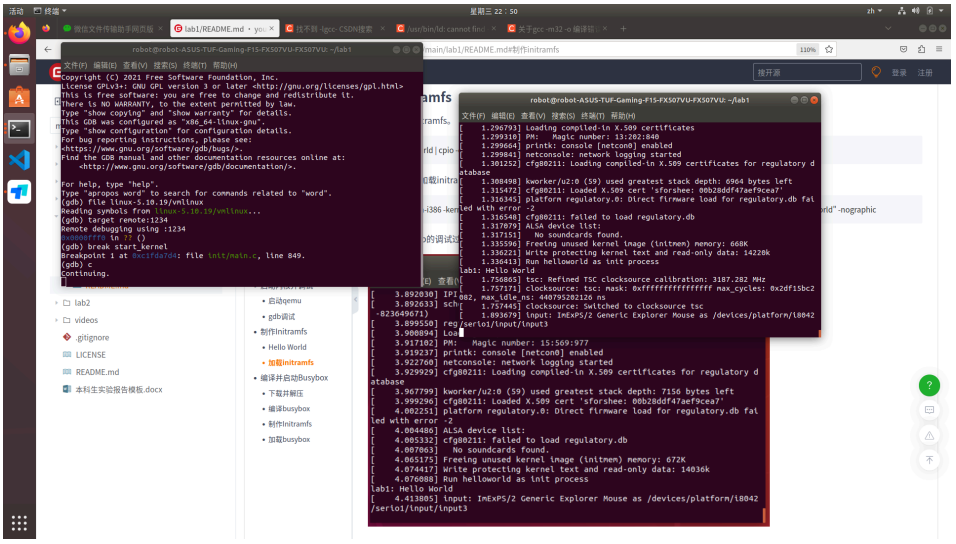
```
cd ~/lab1
qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -initrd initramfs-busybox-x86.cpio.gz -nographic -append "console=ttyS0"
```

然后使用`ls`命令即可看到当前文件夹。

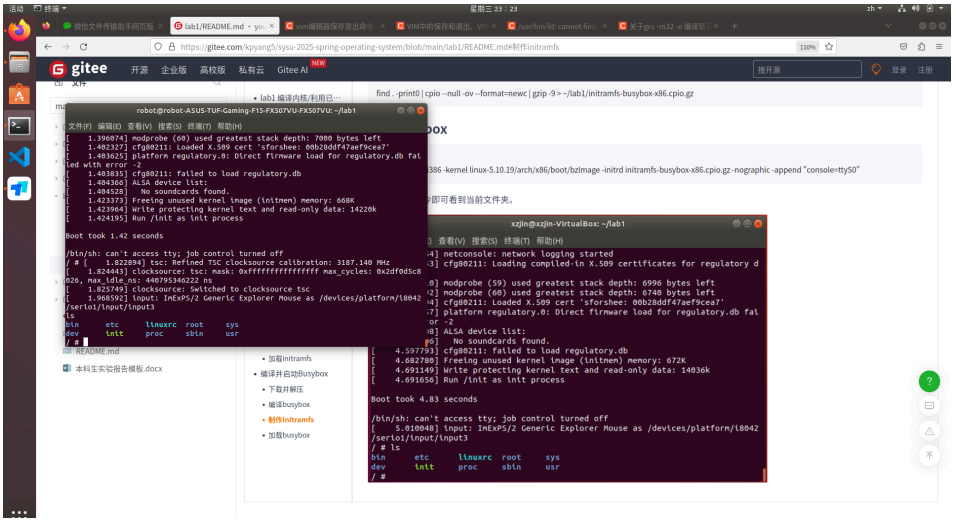


3. 关键代码

4. 实验结果



在ubuntu系统终端安装所有必要工具和环境，并成功在qemu和 gdb 联合下**制作Initramfs 并运行**



编译 Busybox 并启动

5. 总结

本次实验是这学期操作系统原理实验课的第一个实验，总体来看难度不大，耗费的时间和精力较小。个人认为本次实验的关键在于：

正确安装环境

安装环境是开始实验的第一步，也是实验能否进行的决定性步骤。Ubuntu 的系统安装对于入门者来说是一个不小的挑战，个人认为应当结合 CSDN 和 b 站上的教程，根据自己 PC 的型号和各种硬件的标准，选择合适的 Ubuntu 系统安装。个人更推荐使用双系统，能够提供性能上的保障，并且两个系统互不干扰，对于磁盘的管理较为有保障。

熟悉Linux中断命令和实验工具

linux 的主要操作是在终端命令行内完成的，其中大多数命令需要初学者自行熟悉。熟悉之后，才能不复制粘贴，而是自行输入。

实验工具和环境包括 gcc 编译器、qemu、gdb 、vim等，需要正确安装和学会使用。

技术收获

1. 掌握Linux内核交叉编译工具链配置方法
2. 理解initramfs在系统启动中的作用机制
3. 实践基于Busybox的最小化系统构建

问题与改进

1. 依赖冲突问题：通过建立本地软件源快照解决版本兼容性问题
2. 调试效率优化：编写GDB脚本自动化常用调试命令
3. 扩展性提升：可尝试集成systemd等初始化系统

理论联系实际

实验验证了操作系统启动过程中"内核加载→initramfs过渡→用户态初始化"的经典架构，加深了对Linux启动流程（从 `start_kernel` 到 `init` 进程）的理解。